Fletton

Backend System User Manual

Version

Version 1.0

Team Members

Jingli Qin

Haichao Xing

Zeng Zhang

Instructors

Luyao Liu

Mark Licata

Industrial Sponsor

T-Mobile

Submission Date

March 2026

Product Overview

This project is a **cloud-based backend system designed to optimize intelligent delivery dispatching**.

Problem Definition

Last-mile delivery systems plan routes **before departure,** but cannot adapt to **Real-world Execution.**

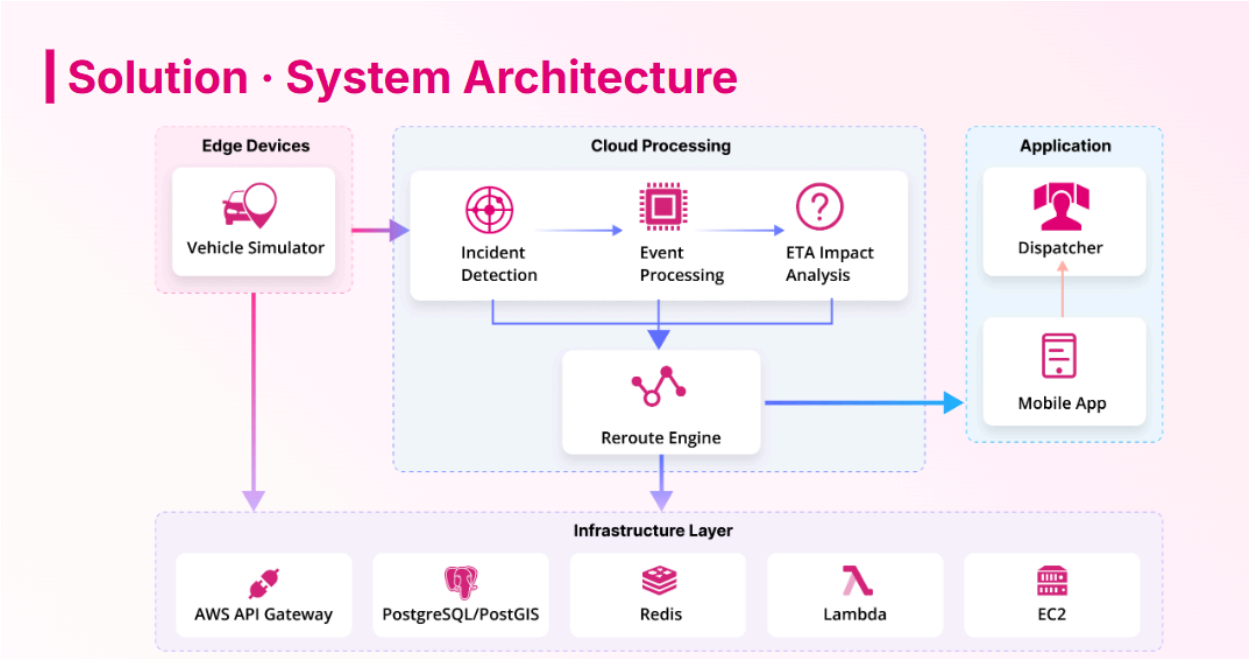
Solution

A T-Mobile 5G–powered system that
detects **on-road incidents** and **dynamically reroutes** deliveries to keep ETAs reliable.

Technical Solution

The system collects **real-time vehicle telemetry data**, including GPS location, vehicle speed, and vehicle status, to monitor abnormal events during the delivery process. These events may include traffic congestion, route deviation, or vehicle breakdown.

Once an incident is detected, the system automatically evaluates its impact on the **Estimated Time of Arrival (ETA)**. If a delay risk is identified, the system triggers a **dynamic routing optimization engine** to recompute the delivery sequence and recommend a new route. This helps reduce delivery delays and improve overall logistics efficiency.



The system is designed for the following users:

- Logistics dispatchers
- Fleet management systems
- Intelligent delivery platforms

Key system features include:

- Real-time vehicle telemetry data ingestion
- Incident detection (traffic jam, vehicle breakdown, route deviation)
- ETA delay impact analysis
- Automatic route resequencing (rerouting)
- Real-time updates of delivery routes and ETA

System Requirements

Supported Operating Systems

The backend system supports the following operating systems:

- Linux (recommended for deployment)
- macOS
- Windows (development environment)

Linux or macOS is recommended for production deployment.

Hardware Requirements

Minimum requirements:

- CPU: 2 vCPU
- Memory: 4 GB RAM
- Storage: 10 GB available disk space

Recommended configuration:

- CPU: 4 vCPU
- Memory: 8 GB RAM

Software Dependencies

The system requires the following software components:

- Python 3.9 or higher
- PostgreSQL 14 or higher

- PostGIS extension
- Redis
- OSRM Routing Engine

Main Python dependencies include:

- psycopg2
- redis
- fastapi
- uvicorn
- requests

Network Requirements

A stable internet connection is required for:

- Vehicle telemetry data uploads
- API requests
- Routing service communication

Recommended bandwidth:

≥ 10 Mbps

Required Accounts

Deployment requires access to the following services:

- AWS account (for Lambda, API Gateway, RDS, and EC2 deployment)
- PostgreSQL database access credentials

Installation and Setup

This section provides step-by-step instructions for installing and configuring the backend system.

Step 1: Clone the Repository

```
git clone https://github.com/your-repository/backend.git
cd backend
```

Step 2: Install Dependencies

```
pip install -r requirements.txt
```

Step 3: Configure Environment Variables

Create a `.env` file:

```
DB_HOST=xxxx
DB_PORT=5432
DB_NAME=delivery_demo
DB_USER=postgres
DB_PASSWORD=password

REDIS_HOST=xxxx
REDIS_PORT=6379

ROUTING_ENGINE_URL=http://localhost:5000
```

Step 4: Initialize the Database

Run the database schema:

```
psql -U postgres -d delivery_demo -f database/schema.sql
```

(Optional) Load seed data:

```
psql -U postgres -d delivery_demo -f database/seed_data.sql
```

Step 5: Start the Backend Server on EC2

uvicorn api.routes:app --reload

After starting the server, access it at:

http://localhost:8000

```
[ec2-user@ip-172-30-1-180 ~]$ uvicorn app:app --host 0.0.0.0 --port 8000
INFO:     Started server process [310283]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO:     172.30.1.117:21096 - "POST /route/evaluate HTTP/1.1" 200 OK
INFO:     172.30.1.117:4000 - "POST /route/evaluate HTTP/1.1" 200 OK
```

Operating Instructions

Start the System

Run the backend API service:

uvicorn api.routes:app --reload

Once started, the system begins listening for incoming vehicle telemetry data.

Start the Vehicle Simulator

The vehicle simulator generates simulated telemetry data and sends it to the backend system.

The system will then:

- Store vehicle location data
- Detect potential incidents
- Calculate ETA impact

```
📍 Loading route from OSRM...
✅ OSRM route loaded: 693 points, baseline duration=2726s
✅ Backed up 10 stops (status + sequence)
📋 Active plan: PLAN_DEMO_20260127
🚀 Simulator running. Sending data every 2 seconds...

[16:19:06] ✅ Dev=0.0m WinDevSec=2 Lat=47.606252, Speed(GPS/CAN)=38.3/38.1, DTCs=[], Task=en_route | ETA(prev/curr/delta)s=2726/5106/2380 | NEXT=STOP_010 eta_to_next=3797s late=248m score=8.2667
shift=7d
[16:19:08] ✅ Dev=0.0m WinDevSec=4 Lat=47.606292, Speed(GPS/CAN)=41.2/42.3, DTCs=[], Task=en_route | ETA(prev/curr/delta)s=5106/4967/-139 | NEXT=STOP_010 eta_to_next=3695s late=246m score=8.2 shi
ft=7d
[16:19:10] ✅ Dev=0.0m WinDevSec=6 Lat=47.606299, Speed(GPS/CAN)=40.7/42.2, DTCs=[], Task=en_route | ETA(prev/curr/delta)s=4967/4870/-97 | NEXT=STOP_010 eta_to_next=3625s late=245m score=8.1667 s
hift=7d
[16:19:12] ✅ Dev=0.0m WinDevSec=8 Lat=47.606316, Speed(GPS/CAN)=41.5/42.2, DTCs=[], Task=en_route | ETA(prev/curr/delta)s=4870/4799/-71 | NEXT=STOP_010 eta_to_next=3574s late=244m score=8.1333 s
hift=7d
[16:19:14] ✅ Dev=0.0m WinDevSec=10 Lat=47.606391, Speed(GPS/CAN)=37.2/36.7, DTCs=[], Task=en_route | ETA(prev/curr/delta)s=4799/4912/113 | NEXT=STOP_010 eta_to_next=3660s late=246m score=8.2 shi
ft=7d

🔍 DEBUG Window: width=30min, predicted=17:19:40, window_end=13:15:00, lateness=245min
[16:19:16] ✅ Dev=0.0m WinDevSec=12 Lat=47.606327, Speed(GPS/CAN)=42.0/41.1, DTCs=[], Task=en_route | ETA(prev/curr/delta)s=4912/4862/-50 | NEXT=STOP_010 eta_to_next=3624s late=245m score=8.1667
shift=7d
[16:19:18] ✅ Dev=0.0m WinDevSec=14 Lat=47.606393, Speed(GPS/CAN)=42.4/40.5, DTCs=[], Task=en_route | ETA(prev/curr/delta)s=4862/4843/-19 | NEXT=STOP_010 eta_to_next=3611s late=245m score=8.1667
shift=7d
[16:19:21] ✅ Dev=0.0m WinDevSec=16 Lat=47.60638, Speed(GPS/CAN)=41.3/41.7, DTCs=[], Task=en_route | ETA(prev/curr/delta)s=4843/4792/-51 | NEXT=STOP_010 eta_to_next=3575s late=244m score=8.1333 s
hift=7d
[16:19:23] ✅ Dev=0.0m WinDevSec=18 Lat=47.606365, Speed(GPS/CAN)=45.5/46.3, DTCs=[], Task=en_route | ETA(prev/curr/delta)s=4792/4624/-168 | NEXT=STOP_010 eta_to_next=3450s late=242m score=8.0667
 shift=7d
[16:19:26] ✅ Dev=799.1m WinDevSec=20 Lat=47.606123, Speed(GPS/CAN)=13.7/13.0, DTCs=[], Task=en_route | ETA(prev/curr/delta)s=4624/5585/961 | NEXT=STOP_010 eta_to_next=4184s late=255m score=8.5 s
hift=7d | EVENT=ROUTE MISMATCH sev=MEDIUM eta impact min=23 reorder=True incident id=f5a534c2-4b13-415c-b40f-32f37d7d8b69
```

View Rerouting Results

When the system detects significant delivery delays:

1. The routing engine calculates a new route.
2. The backend updates the database.
3. The updated route and ETA are pushed to the dispatch system.

Dispatchers can view:

- Current vehicle location
- Detected incident type
- Recommended new route
- Updated ETA

```
[16:19:21] ✅ Dev=0.0m WinDevSec=16 Lat=47.60638, Speed(GPS/CAN)=41.3/41.7, DTCs=[], Task=en_route | ETA(prev/curr/delta)s=4843/4792/-51 | NEXT=STOP_010 eta_to_next=3575s late=244m score=8.1333 s
hift=7d
[16:19:23] ✅ Dev=0.0m WinDevSec=18 Lat=47.606365, Speed(GPS/CAN)=45.5/46.3, DTCs=[], Task=en_route | ETA(prev/curr/delta)s=4792/4624/-168 | NEXT=STOP_010 eta_to_next=3450s late=242m score=8.0667
 shift=7d
[16:19:26] ✅ Dev=799.1m WinDevSec=20 Lat=47.606123, Speed(GPS/CAN)=13.7/13.0, DTCs=[], Task=en_route | ETA(prev/curr/delta)s=4624/5585/961 | NEXT=STOP_010 eta_to_next=4184s late=255m score=8.5 s
hift=7d | EVENT=ROUTE_MISMATCH sev=MEDIUM eta_impact_min=23 reorder=True incident_id=f5a534c2-4b13-415c-b40f-32f37d7d8b69

🚨 INCIDENT DETECTED! severity=MEDIUM, lateness=255min, deviation_score=8.5

📦 BEFORE REROUTE (incident_id=f5a534c2-4b13-415c-b40f-32f37d7d8b69):
  STOP_010: seq=1 status=PENDING
  STOP_009: seq=2 status=PENDING
  STOP_008: seq=3 status=PENDING
  STOP_007: seq=4 status=PENDING
  STOP_006: seq=5 status=PENDING
  STOP_005: seq=6 status=PENDING
  STOP_004: seq=7 status=PENDING
  STOP_003: seq=8 status=PENDING
  STOP_002: seq=9 status=PENDING
  STOP_001: seq=10 status=IN_PROGRESS
  📤 Sending to Lambda:
    - vehicle_id: TRUCK_001
    - eta_sec: 5585
    - location: {'lat': 47.606123, 'lon': -122.322114}
    - num_stops: 10
    - lateness_min: 255
  🔄 [REROUTE] ✅ decision=REROUTE stop_order=['STOP_001', 'STOP_002', 'STOP_003', 'STOP_004', 'STOP_005', 'STOP_006', 'STOP_007', 'STOP_008', 'STOP_009', 'STOP_010']
  ✅ [DB UPDATED] Applied new current_sequence order
```

Shut Down the System

Stop the backend service:

CTRL + C

Technical Specifications

System Architecture

The system follows a **cloud-based event-driven architecture**:

Vehicle Simulator → API Gateway → Lambda → Database → Routing Engine → Frontend

Technologies Used

Backend

- Python
- FastAPI
- AWS Lambda
- AWS API Gateway

Database

- PostgreSQL
- PostGIS

Cache

- Redis

Routing

- OSRM Routing Engine

Storage

- AWS S3

Database Structure

The core database tables include:

- vehicles
- delivery_plans
- delivery_stops
- incidents
- stop_resequence_results

The database supports **spatial queries and route analysis** using PostGIS.

Security and Privacy

Because the system processes vehicle location data, several privacy and security considerations must be addressed.

Data Privacy

- The system does not store personal user information.
- Only vehicle-level telemetry data is collected and processed.

Data Security

- API communication is secured using HTTPS.
- Database access is restricted within a private VPC network.

Known Security Limitations

- The demo environment uses simulated data.
- Full authentication and authorization mechanisms are not implemented.
- Production-grade monitoring and logging systems are not yet integrated.

Troubleshooting

Backend Fails to Start

Check the following:

- Python version compatibility
- Dependency installation
- Port availability

Database Connection Failure

Verify:

- Database host address
- Username and password
- Network access permissions

Routing Calculation Failure

Check:

- Whether the OSRM service is running
- Whether the routing engine URL is correctly configured

Limitations

The current system has the following limitations:

- Vehicle data is simulated rather than collected from real vehicles
- The routing engine uses a simplified optimization algorithm
- Large-scale load testing has not been performed
- Production monitoring systems are not fully implemented
- Redis caching strategy is still basic

Future improvements may include:

- Integration with real vehicle telemetry devices
- Enhanced routing optimization algorithms
- Improved system scalability
- Multi-vehicle delivery optimization support